

1Aim: Implement BFS on a 3×3 grid to find the path from start to goal.

CODE

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def bfs(start):
    visited = []
    queue = deque([start])

    while queue:
        node = queue.popleft()

        if node not in visited:
            print(node, end=" ")
            visited.append(node)
            queue.extend(graph[node])

bfs('A')
```

2Aim: Implement DFS traversal on a small graph.

CODE

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()

    print(start)
    visited.add(start)

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

dfs(graph, 'A')
```

5Aim: Implement A* search to find the optimal path on a 4×4 grid.

CODE

```
from heapq import heappush, heappop

graph = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['E'],
    'D': [],
    'E': []
}

h = {
    'A': 3,
    'B': 2,
    'C': 4,
    'D': 1,
    'E': 0
}

cost = {
    ('A', 'B'): 1,
    ('A', 'C'): 3,
    ('B', 'D'): 2,
    ('C', 'E'): 2
}

pq = []
heappush(pq, (h['A'], 'A', 0))

visited = set()

while pq:
    f, n, g_n = heappop(pq)

    if n in visited:
        continue

    visited.add(n)
    print(n, end=' ')

    if n == 'E':
        break

    for x in graph[n]:
        new_g = g_n + cost[(n, x)]
        new_f = new_g + h[x]
        heappush(pq, (new_f, x, new_g))
```

7Aim: Solve the N-Queens problem using backtracking (Constraint Satisfaction Problem).

CODE

```
n = 4
x = [-1] * n

def safe(k):
    for i in range(k):
        if x[i] == x[k] or abs(x[i] - x[k]) == abs(i - k):
            return 0
    return 1

def show():
    for i in range(n):
        for j in range(n):
            print("Q" if x[i] == j else ".", end=" ")
        print()
    print()

def solve(k):
    if k == n:
        show()
        return

    for i in range(n):
        x[k] = i
        if safe(k):
            solve(k + 1)
```

solve(0)

9Aim: Apply optimization techniques to find the maximum value in a list.

CODE

```
a=[3,5,2,9,1]
```

```
print(max(a))
```

10Aim: Represent and evaluate propositional logic expressions.

CODE

```
def eval_logic(p, q):
    def show_logic(p, q):
        print("p =", p, "q =", q)
        print("p and q =", p and q)
        print("p or q =", p or q)
        print("NOT p =", not p)
        print("NOT q =", not q)
        print("p -> q =", (not p) or q)
        print("p <-> q =", p == q)
        print()

    for p in [True, False]:
        for q in [True, False]:
            show_logic(p, q)
```

eval_logic(True, True)

11Aim: Implement a basic rule-based expert system for weather classification.

CODE

```
def weather(temp, humidity):
    if temp > 30 and humidity < 50:
        return "Hot and Dry"
    elif temp > 30 and humidity >= 50:
        return "Hot and Humid"
    elif temp <= 30 and humidity >= 50:
        return "Cool and Humid"
    else:
        return "Cool and Dry"

t = int(input("Enter temperature: "))
h = int(input("Enter humidity: "))
```

```
print("Weather:", weather(t, h))
```

12Aim: Implement a basic AI agent with simple decision-making rules.

CODE

```
def weather(temp, humidity):
    if temp > 30 and humidity < 50:
        return "Hot and Dry"
    elif temp > 30 and humidity >= 50:
        return "Hot and Humid"
    elif temp <= 30 and humidity >= 50:
        return "Cool and Humid"
    else:
        return "Cool and Dry"

t = int(input("Enter temperature: "))
h = int(input("Enter humidity: "))
```

```
print("Weather:", weather(t, h))
```

13Aim: Implement a basic rule-based chatbot.

CODE

```
while True:
    msg = input("You: ").lower()
    if msg == "hi" or msg == "hello":
        print("Bot: Hello!")
    elif "how are you" in msg:
        print("Bot: I'm fine")
    elif "name" in msg:
        print("Bot: I'm a simple chatbot")
    elif msg == "bye":
        print("Bot: Goodbye!")
        break
    else:
        print("Bot: I don't understand")
```